Codebase, Formulas, And One-Go Runbook

This file explains how the codebase is organized, how the formulas in the report are implemented, and how to run the complete project.

Folder Structure

Path	Purpose
`backend/app/config.py`	Central configuration: paths, battery parameters, PV capacity, load limits, timezone.
`backend/app/data/cleaning.py`	NASA POWER loading, timestamp conversion, cleaning, normalization, and weather plots.
`backend/app/data/synthetic.py`	Synthetic load, tariff, PV power, battery SoC simulation, and derived features.
`backend/app/ml/lstm.py`	GPU LSTM training for solar and load forecasting.
`backend/app/ml/inference.py`	Forecast API payload logic using live simulator, LSTM artifacts, or persistence fallback.
`backend/app/rl/microgrid_env.py`	Custom PPO environment with battery physics and reward function.
`backend/app/rl/train_ppo.py`	Stable-Baselines3 PPO training pipeline.
`backend/app/services/dispatch.py`	Operator-readable dispatch decision table.
`backend/app/services/metrics.py`	Cost, renewable utilization, grid dependency, self-sufficiency, plots.
`backend/app/services/alerts.py`	Real-time risk detection.
`backend/app/services/live_simulator.py`	Dynamic backend telemetry generator for live dashboard mode.
`backend/app/services/scenarios.py`	Normal, peak load, low solar, and tariff spike simulation.
`backend/app/services/exports.py`	CSV/MAT export for MATLAB/Simulink workflow.
`backend/app/main.py`	FastAPI app and endpoint definitions.
`frontend/src/App.jsx`	Main React operator dashboard.
`frontend/src/api/client.js`	API client, endpoint calls, API fallback, cache fallback.

Core Configuration

Defined in `backend/app/config.py`.

Parameter	Value
PV capacity	140 kW
PV performance ratio	0.82
Load min	24 kW
Load max	175 kW
Peak load risk	145 kW
Battery capacity	180 kWh
Battery min SoC	20 percent
Battery max SoC	90 percent
Initial SoC	55 percent
Round-trip efficiency	0.93
Max charge	55 kW
Max discharge	58 kW
Local timezone	Asia/Kolkata

Data Cleaning Formula And Logic

File: `backend/app/data/cleaning.py`

NASA POWER columns are standardized:

NASA column	Project column
`ALLSKY_SFC_SW_DWN`	`ghi`
`ALLSKY_SFC_SW_DNI`	`dni`
`ALLSKY_SFC_SW_DIFF`	`diffuse_irradiance`
`T2M`	`temperature_c`
`WS10M`	`wind_speed_mps`
`RH2M`	`humidity_pct`
`PRECTOTCORR`	`precipitation_mm`

Cleaning steps:

1. Read CSV after NASA POWER metadata header.
2. Replace NASA missing code `-999` with null.
3. Build UTC timestamp from year, month, day, hour.
4. Convert UTC timestamp to Asia/Kolkata local timestamp.
5. Drop duplicate timestamps.

6. Sort by time.
7. Interpolate numeric weather values by time.
8. Forward/backward fill any edge missing values.
9. Clip values to realistic physical ranges.
10. Save cleaned CSV, normalized CSV, scaler, report, and plots.

Clipping ranges:

Feature	Range
GHI	0 to 1100
DNI	0 to 1100
Diffuse irradiance	0 to 900
Temperature	-5 C to 55 C
Wind speed	0 to 35 m/s
Humidity	0 to 100 percent
Precipitation	0 to 300 mm

PV Generation Formula

File: `backend/app/data/synthetic.py`

The code computes PV output from GHI:

```
solar_kw = (ghi / 1000) * pv_capacity_kw * pv_performance_ratio
```

Then it applies derating:

```
temperature_derate = 1 - max(temperature_c - 25, 0) * 0.004
humidity_derate    = 1 - max(humidity_pct - 85, 0) * 0.001

solar_kw = solar_kw * clipped_temperature_derate * clipped_humidity_derate
```

Then it enforces:

```
0 <= solar_kw <= pv_capacity_kw
```

Why this matters:

- Solar panels lose output at high temperature.
- High humidity/cloud conditions reduce usable generation.
- Physical clipping prevents impossible PV values.

Synthetic Load Formula

File: `backend/app/data/synthetic.py`

Load is generated using a realistic demand shape:

```
load =
  night_base
  + daytime_activity
  + morning_peak
  + evening_peak
  + cooling_load
  + humidity_load
```

The shape components are:

```
morning_peak = 34 * exp(-0.5 * ((hour - 8) / 1.7)^2)
evening_peak = 48 * exp(-0.5 * ((hour - 20) / 1.9)^2)
daytime_activity = 32 * exp(-0.5 * ((hour - 14) / 4.2)^2)
```

Season and weekday effects:

```
summer_factor = 1.16 for March to June
winter_factor = 0.91 for November to February
weekend_factor = 0.88 for Saturday and Sunday
```

Weather-driven load:

```
cooling_load = max(temperature_c - 28, 0) * 2.2
humidity_load = max(humidity_pct - 80, 0) * 0.18
```

Noise and event spikes:

```
noise = Gaussian noise with mean 0 and standard deviation 3.5
event_spike_probability = 1.2 percent
event_spike_magnitude = 12 to 28 kW
```

Final constraint:

```
24 kW <= load_kw <= 175 kW
```

Current observed output:

Metric	Value
Min load	26.383 kW
Max load	151.913 kW
Mean load	73.084 kW
Within limits	True

Tariff Formula

File: `backend/app/data/synthetic.py`

India-style time-of-use tariff:

```
if hour in 22:00 to 05:59:
    tariff = 2.6 INR/kWh
elif hour in 18:00 to 21:59:
    tariff = 9.2 INR/kWh
else:
    tariff = 5.6 INR/kWh
```

Why this matters:

- Battery should charge during low-price hours or solar surplus.
- Battery should discharge during evening peak tariff.
- No random tariff jumps are used.

Battery Energy And SoC Formulas

Files:

- `backend/app/data/synthetic.py`
- `backend/app/services/dispatch.py`
- `backend/app/services/live_simulator.py`
- `backend/app/rl/microgrid_env.py`

Energy bounds:

```
min_energy_kwh = capacity_kwh * min_soc_pct / 100
max_energy_kwh = capacity_kwh * max_soc_pct / 100
```

With current config:

```
min_energy_kwh = 180 * 20 / 100 = 36 kWh
max_energy_kwh = 180 * 90 / 100 = 162 kWh
```

Charge efficiency and discharge efficiency:

```
eta_charge = sqrt(roundtrip_efficiency)
eta_discharge = sqrt(roundtrip_efficiency)
```

With round-trip efficiency 0.93:

```
eta_charge = eta_discharge = sqrt(0.93)
```

Charging update:

```
energy_next = energy_now + charge_kw * eta_charge * delta_t
```

Discharging update:

```
energy_next = energy_now - discharge_kw / eta_discharge * delta_t
```

SoC:

```
soc_pct = energy_kwh / capacity_kwh * 100
```

The code clips energy to the safe battery band, so:

```
20 percent <= SoC <= 90 percent
```

Current validation:

Metric	Value
SoC min observed	20 percent
SoC max observed	90 percent
Battery violations	0
Within safe SoC	True

Dispatch Calculation

File: `backend/app/services/dispatch.py`

The dispatch engine creates the operator decision table.

For each timestep:

```
surplus_kw = solar_kw - load_kw
```

If solar surplus exists:

```
if surplus_kw > 5 and battery has room:
    action = charge
```

If tariff or demand risk is high:

```
if tariff >= 8 or load_kw >= peak_load_risk_kw:
    if load is greater than solar and battery has energy:
        action = discharge
```

If off-peak tariff and future peak is expected:

```
if action is idle and tariff <= 3 and future peak need exists:
    action = charge
```

Grid import:

```
grid_kw = max(load_kw - solar_kw - discharge_kw + charge_kw, 0)
```

In the code:

```
battery_power_kw > 0 means discharge
battery_power_kw < 0 means charge
```

Renewable used:

```
renewable_used_kw = min(solar_kw, load_kw + charge_kw)
```

Curtailment:

```
curtailed_kw = max(solar_kw - renewable_used_kw, 0)
```

Cost:

```
cost_inr = grid_kw * tariff_inr_kwh
```

Each row also gets a human-readable reason, for example:

- Solar surplus available; charging BESS without grid import.
- Peak tariff or high demand; discharging BESS to reduce grid purchase.
- Off-peak tariff; pre-charging BESS for peak-period cost reduction.
- Hold battery inside safe band; no economic dispatch needed.

Performance Metrics Formulas

File: `backend/app/services/metrics.py`

Baseline grid import:

```
baseline_grid_kw = max(load_kw - solar_kw, 0)
```

Baseline cost:

```
baseline_cost = sum(baseline_grid_kw * tariff)
```

Optimized cost:

```
optimized_cost = sum(dispatch_grid_kw * tariff)
```

Cost savings:

```
savings = baseline_cost - optimized_cost
savings_pct = savings / baseline_cost * 100
```

Renewable utilization:

```
renewable_utilization_pct = renewable_used / total_solar * 100
```

Grid dependency:

```
grid_dependency_pct = total_grid / total_load * 100
```

Renewable share:

```
renewable_share_pct = renewable_used / total_load * 100
```

Self-sufficiency:

```
self_sufficiency_pct = 100 - grid_dependency_pct
```

Current seven-day result:

Metric	Value
Baseline cost	INR 41,165.47
Optimized cost	INR 36,130.22
Savings	INR 5,035.25
Savings percent	12.23 percent
Renewable utilization	93.37 percent
Grid dependency	68.58 percent
Self-sufficiency	31.42 percent

PPO Environment Reward Formula

File: `backend/app/rl/microgrid_env.py`

The environment state is normalized and contains:

```
[solar, load, battery_energy, tariff, hour, hour_sin, hour_cos, is_weekend]
```

Current action space:

```
0 = idle
1 = charge
2 = discharge
```

Reward:

```
reward =
  - cost_inr / 24
  - degradation_penalty
  - peak_penalty
  - violation_penalty
  + renewable_reward
```

Where:

```
degradation_penalty = abs(battery_power_kw) * degradation_cost_inr_per_kwh
peak_penalty = 0.06 * grid_kw if tariff >= 8 else 0
violation_penalty = 45 if action violates battery feasibility else 0
renewable_reward = 0.018 * renewable_used_kw
```

This teaches the agent to:

- Reduce import cost.
- Avoid high-tariff grid import.
- Avoid invalid battery actions.
- Prefer renewable usage.
- Avoid excessive battery cycling.

LSTM Forecasting Architecture

File: `backend/app/ml/lstm.py`

Important settings:

Parameter	Value
Sequence length	48 timesteps
Hidden size	96
LSTM layers	2
Dropout	0.15
Optimizer	AdamW
Loss	MSE

Parameter	Value
Device	CUDA GPU

The model receives a sequence:

```
X[t-47], X[t-46], ..., X[t]
```

It predicts:

```
y[t+1]
```

Targets:

- `solar_kw`
- `load_kw`

API Endpoints

File: `backend/app/main.py`

Endpoint	Purpose
`GET /health`	Backend status and GPU summary.
`GET /forecast`	Solar/load forecast records.
`GET /optimize`	Recommendation and dispatch schedule.
`GET /decisions`	Operator decision table.
`GET /metrics`	Cost, sustainability, battery metrics.
`GET /alerts`	Active risk alerts.
`POST /simulate`	Runs normal, peak load, low solar, or tariff spike scenario.
`GET /matlab/export`	Exports CSV/MAT files for MATLAB/Simulink.
`GET /live/status`	Live simulator status.
`POST /live/start`	Starts dynamic telemetry generation.
`POST /live/stop`	Stops dynamic telemetry generation.
`POST /live/override`	Sets auto, force charge, force discharge, or island mode.

One-Go Run

Use this from the project root:

```
cd "C:\Users\ayush\Desktop\Final Year Project"
.\start_system.bat
```

This starts:

1. FastAPI backend using the project venv.
2. React frontend using Vite.

Open:

```
http://127.0.0.1:5173
```

API docs:

```
http://127.0.0.1:8000/docs
```

Manual Run

Backend:

```
cd "C:\Users\ayush\Desktop\Final Year Project"
venv\Scripts\python.exe -m uvicorn backend.app.main:app --host 127.0.0.1 --port
8000
```

Frontend:

```
cd "C:\Users\ayush\Desktop\Final Year Project\frontend"
npm.cmd run dev
```

Pipeline And Training Commands

Run data pipeline:

```
venv\Scripts\python.exe backend\scripts\run_pipeline.py
```

Train LSTM models on GPU:

```
venv\Scripts\python.exe backend\scripts\train_forecasts.py --epochs 8 --batch-size
```

```
256 --sequence-length 48
```

Train PPO:

```
venv\Scripts\python.exe backend\scripts\train_ppo.py --timesteps 20000
```

Generate EDA and report audit:

```
venv\Scripts\python.exe backend\scripts\audit_report_and_generate_eda.py
```

Export MATLAB/Simulink files:

```
venv\Scripts\python.exe backend\scripts\export_simulink.py
```

Run tests:

```
venv\Scripts\python.exe -m pytest backend\tests -q
```

Important Run Rule

All Python commands must use:

```
venv\Scripts\python.exe
```

Do not use global Python or Anaconda Python for backend execution, training, or tests. This keeps CUDA PyTorch, FastAPI, Stable-Baselines3, pandas, and all backend dependencies isolated inside the project venv.